

ArtHound

ArtHound is a canonical production data layer for game studios. It replicates source tool data (Airtable, Jira, and future connectors) into a central Supabase database, then all product features (estimation, scheduling, vendor dispatch, reviews) read from that database rather than from the source tools directly. Studios connect their existing pipelines, and ArtHound becomes the stable cross-tool identity and coordination layer sitting on top.

The core conceptual model is **PAW: Product, Asset, Work**. Every piece of data belongs to one of these three tiers, and every record in the system links back to a canonical asset that acts as the stable thread connecting internal production data, vendor deliveries, reviews, and generated work into a single coherent record.

Architecture Overview



Studios onboard by connecting their source tool through an init wizard, defining their P→A→W hierarchy, and mapping source fields to ArtHound's schema. Once that first sync runs, all product features are available without touching the source tool again.

Security Architecture

ArtHound holds unreleased game IP for multiple studios and vendors in one database, so tenant isolation is the central security property. Isolation is enforced in Postgres itself through Row-Level Security (RLS), not only in application code, so a missing filter in a route handler cannot leak one org's data to another.

Four runtime identities

Every database call runs as exactly one of four identities. The service-role key, which bypasses RLS, is kept out of all request paths.

1. **User** (anon key + the caller's Supabase JWT). The default for request handlers. RLS policies scope every read and write to the user's own org. The request's token is bound to a `ContextVar` for the

duration of the request, so every query made while serving it runs as that user.

2. **System** (`arthound_system` , a narrow Postgres role). Background jobs (sync, polling, the attachment copy worker, token refresh, log trimming) run here via `system_identity()` . It is least-privilege and scoped by its own policies, not god-mode. The token is a short-lived JWT minted with the `arthound_system` role claim, which PostgREST maps to the Postgres role. It is signed with a dedicated **ES256** key we control — imported into the project's JWKS as a verify-only **standby** key (it signs nothing in GoTrue, so user tokens are untouched) — so the system identity does not depend on the legacy shared HS256 secret that Supabase is retiring. Unsetting the signing-key env var falls the minter back to the legacy secret, which is the instant rollback.
3. **RPC** (`SECURITY DEFINER` functions owned by `arthound_rpc`). Cross-org writes that no single org's RLS could authorize (accepting a link invite, ingesting a payload, freezing a shared estimate) go through these functions. Authorization lives inside the function body and is keyed on `auth.uid()` , never on a value the caller passes in.
4. **service_role** (migrations, plus two structural carve-outs). Reserved for schema migrations and break-glass. The two sanctioned exceptions are the GoTrue Admin API (user creation and email lookups, which are not PostgREST tables) and Storage byte serving (authorized by a user-context metadata check before any byte is fetched).

The cutover from the old service-role-everywhere model to RLS is controlled by a single runtime flag, `USE_USER_IDENTITY` , read per request. With the flag off, `_headers()` returns service-role headers and behavior is identical to the pre-RLS app. With it on, `_headers()` returns the bound user or system token and fails closed if no identity is present, rather than silently falling back to service-role. Because the flag is a runtime value and not a deploy, the cutover and its rollback are instant and independent of code releases.

Predicate functions and tenancy patterns

Membership is resolved through a small set of `SECURITY DEFINER` , `STABLE` , `SET search_path = ''` helper functions (`current_studio_ids()` , `current_vendor_ids()` , `is_my_org()` , `is_org_admin()` , `is_link_party()`). Policies call these helpers instead of querying membership tables directly, which removes the need to grant users any direct read on the membership tables and keeps the Postgres role identity, not a spoofable JWT claim, as the source of truth.

Roughly fifty tables are covered by six tenancy patterns: owner-record, single-org, polymorphic (`owner_type` + `owner_id`), dual-party (studio and vendor both see a shared row, but only the owner writes it), grant-based, and system-only (deny-all to users, written only by the system role or an RPC). FORCE RLS is applied broadly, with a deliberate exemption for the three membership tables the predicate functions read, to avoid a recursion where a policy consults a helper that is itself subject to the same policy.

Storage and the byte layer

Supabase Storage is a separate service with its own access control, so the table-level cutover does not by itself secure attachment bytes. Every byte-serving and upload path first performs a user-context metadata read (of `replicated_assets` , `payload_dispatches` , or `review_attachments`) that RLS

scopes to the caller's org. If that row is invisible to the caller the request returns 404 before any byte is fetched. The byte transfer itself stays service-role, since access was already authorized.

Test guard

`scripts/rls_persona_matrix.py` is a standalone correctness guard that mints a JWT per persona and asserts the row-visibility contract for every tenancy-critical table by hitting PostgREST directly, so it exercises the live policies regardless of the application flag. It checks own-org visibility for studios and vendors, cross-org denial, the link-authorized counterparty read, the system identity, anon deny-all, and that no route imports the break-glass path. Keep it green before any policy change ships. The full design and migration history live in [docs/plans/rls-migration.md](https://docs.art-hound.com/plans/rls-migration.md).

External agents (MCP)

External AI agents reach ArtHound through an MCP server (`/mcp`), not the REST API — but an agent is **not a fifth identity**. It authenticates with an API key that resolves to a dedicated service-account member (`member_role='agent'`) of one org; the server mints a short-lived user token for that principal, so every tool call runs under the **User** identity and the same RLS as a human member, never service-role. Within-org scope (read vs write, tool allowlist) and per-credential rate limits are enforced at the app layer, and writes carry mandatory actor attribution. See [docs/mcp.md](https://docs.art-hound.com/mcp.md).

`scripts/test_agent_scoping.py` guards the cross-org scoping.

Documentation

Sync Layer

The sync layer is the most important architectural component. It runs continuously, triggered on login, by a polling loop, and by source tool webhooks, and keeps the Supabase `replicated_*` tables current. A sync run proceeds through six phases: init (load credentials and mappings), fetch (pull raw records from the source tool), normalize (map source fields to ArtHound slots), diff (detect changes by source hash), write (batch upsert to Supabase), and cleanup (delete orphaned records). Two connectors are currently implemented, Airtable and Jira Cloud/Data Center, against a shared `BaseConnector` interface. Delta syncs use per-owner cursors so only changed records are written on subsequent runs.

Studio Onboarding

A gated five-step wizard that connects a studio's source tool and runs the first full sync. Steps in order: validate and encrypt source credentials, fetch and cache the full source schema, define the P→A→W hierarchy (which source tables are Products, Assets, and Work, and how they link), map source fields to ArtHound slots and classify them by display tier, then start the background init job. Each step gates the next. A polling endpoint reports job progress so the UI can show live status. New accounts create their org through a post-login onboarding step rather than at signup, so org creation runs under the new account's own identity.

Asset Viewer

The central UI surface: a three-panel layout with a filterable asset list on the left and a tabbed detail panel on the right. Tabs are Details (slot fields plus meta fields grouped by display tier, with a "Show more" collapse for secondary-tier fields), Work (source work vs ArtHound-generated work, with a Gantt timeline view), Reviews, Attachments, and Bugs. Column visibility in the list panel is persisted in localStorage. Field display tier (primary, secondary, hidden) is configured per studio in the field mapping UI and drives both the list columns and the detail panel. Schema drift, meaning changes to the source tool's field schema, surfaces as a banner and badge prompting the studio to review their mappings.

Estimation Engine

An organisation defines a library of **workflow steps** (production tasks with optional craft labels and dependency edges), then chooses **variable fields** from their source schema (for example "Asset Type" or "Complexity"). The system enumerates all unique combinations of those field values across synced assets and builds an estimate matrix: one cell per (workflow step x variable combination), each holding a day count. The matrix is filled via an inline spreadsheet UI. When the scheduler generates work for an asset, it resolves the asset's variable values against this matrix to produce step-level estimates. Dependency edges between steps are validated server-side so a circular dependency can never be saved. The stack is org-scoped, owned by either a studio or a vendor.

Vendor Estimate Sharing

Vendors maintain their own estimation matrix, optionally vary their rates per studio relationship (a base matrix overlaid with per-link overrides), and share a **frozen, granularity-controlled snapshot** with a linked studio, the reverse direction of asset payload dispatch. At share time the vendor chooses how much process detail to expose (asset total, by craft, or per workflow step), and the projector enforces that boundary so internal process detail never leaks beyond the chosen level. Re-sharing replaces the prior share in that channel, so dialing disclosure down genuinely reduces what the studio can see. Delivery is a route-scoped inbox with revoke, optional expiry, and an append-only access log. v1 is visible-only on the studio side and not yet wired into scenario planning.

Schedule and Generated Work

The scheduler generates work snapshots from the estimate matrix and writes them back to the studio's source tool (creating Jira issues or Airtable records with link-back fields). **Source work** (`replicated_work`) is what the source tool already contains; **generated work** (`generated_work`) is what ArtHound proposes. Generated work is soft-deleted rather than hard-deleted so historical snapshots are available for variance analysis. A reconcile endpoint soft-deletes generated work rows whose source records have since been removed.

NumberBot

An in-app AI assistant powered by Claude Haiku. Before each turn it fetches live context (asset inventory, field mappings, source work, generated work, and reviews), builds an ASCII summary, and passes it to the model with prompt caching enabled. Scope is strictly limited to production data

questions: the assistant refuses general knowledge, business advice, and anything not grounded in the fetched context. It works for both studio and vendor sessions; vendor context is scoped to dispatched assets only.

Studio/Vendor Handshake

The prerequisite gate for payload dispatch. Studios send invites to vendors by searching their unique handle; vendors preview the studio's payload templates and accept, triggering the creation of an active link and an optional field mapping setup step. Either party can cancel a link, which immediately revokes all outstanding (non-ingested) dispatches and writes a full audit trail. Cross-org review behaviour is configured on the link via its review protocol (see Asset Reviews); the legacy `review_collaboration_mode` field set at invite time is vestigial and unread.

Asset Payload Dispatch

Studios dispatch frozen snapshots of asset data to connected vendors. Each dispatch is immutable after creation, so changes to the studio's source data do not affect what the vendor sees. Vendors map payload fields to their own source tool schema and ingest, creating a real Jira issue or Airtable record in their own tool. ArtHound writes a canonical link (`payload_export_records`) back to the studio's asset on successful ingest, making the vendor's record permanently traceable. Dispatches can be revoked by the studio at any time, and an expiry window is set at dispatch time. An append-only audit log tracks every access event for both parties.

Attachment Architecture

Attachments from source tools (images, video, PDFs, documents) are surfaced inline in the UI via a content-addressed storage layer in Supabase Storage. There are two copy triggers: **copy-on-dispatch** (attachments are downloaded and stored when a studio dispatches an asset to a vendor, freezing the snapshot) and **copy-on-first-view** (studio attachments are copied the first time a user opens them in the Asset Viewer). Every blob is stored at `attachments/sha256/{hex_hash}` , making deduplication free and paths collision-safe. A background drain loop processes the copy job queue every 30 seconds, and a nightly purge removes orphaned blobs. Browser media components (`<img>` , `<video>` , `pdf.js`) receive blob URLs created from authenticated proxy responses, since browsers cannot send JWT headers in media element requests.

Asset Reviews

ArtHound-native records attached to canonical assets — the method and record of sign-off and delivery. They are not synced to or from any source tool and exist only in ArtHound's database. Reviews carry threaded comments with visibility lanes (internal/shared, default private), file attachments, and an append-only audit trail. Vendors keep fully private internal reviews and **promote** them across the org wall as trimmed copies (field/comment/attachment selection, payload-template style, saveable as templates); ad-hoc cross-org review requests are always available on an active link. Studios define **required submissions** per link via an ordered review protocol, which vendors see as a computed per-asset checklist and fulfil by tagged submissions. Formal delivery ends in the studio **accepting** a review, which freezes an immutable snapshot (shared-lane content + asset data + attachment refs); re-delivery

chains as new linked revisions. Enforcement is user-context RLS plus `SECURITY DEFINER` RPCs for every cross-org mutation.

Member Management and Org Hub

Studios and vendors are multi-user organisations with three membership tiers: `owner`, `admin`, and `user`. New members join via an 8-character invite code and land in a pending state until an org admin approves their join request. The org hub shows the full member list with inline role management, the current invite code (with regeneration), and pending join requests. Ownership transfer is atomic via a Postgres RPC, so there is always exactly one owner. A 15-second membership cache means role changes propagate within 15 seconds.

Scenario Planner

Studios model hypothetical production schedules ("when can we ship?" or "what can we complete by date X?") without touching their source tool. A Claude Haiku scoping conversation gathers planning parameters (release cadence, asset counts by classification, optional per-craft concurrency caps), then a generation engine writes an ephemeral product/asset/work plan to session-scoped tables. Two engines are available: a deterministic rule-based engine (no AI, fully reproducible, grounded entirely in the studio's estimate matrix and workflow graph) and a Claude Sonnet multi-pass engine. Both support `earliest_ship` (schedule forward, derive completion date) and `target_date` (schedule backward, flag infeasibility) modes. The rule-based engine uses a DAG-aware, asset-at-a-time scheduler with dependency-respecting date placement, a two-pass capped scheduler (forward cap-push plus backward pullback for uncapped steps), and a cadence scheduling path that derives sprint release dates from actual asset completions. After generation, Haiku answers questions about the plan in a discussion chat. Export to CSV and Write to Source are deferred.

LoreBot

A proof-of-concept document-reading assistant. Given a canonical asset (studio) or a dispatch (vendor), LoreBot reads the attached files from Supabase Storage (PDFs, text, images) and answers questions about their content using Claude Haiku with vision. PDFs are extracted via `pypdf`, and up to four images are passed as base64 vision blocks. Attachments must be copied to Storage before chat begins; a replication endpoint triggers the copy synchronously. Prompt caching is applied to the attachment context. Explicitly marked PoC, not for use with confidential data.

MCP Server

Exposes ArtHound's canonical production data to external AI agents over the Model Context Protocol, mounted at `/mcp` (FastMCP, Streamable HTTP). Agents authenticate with an API key that maps to a service-account member of one org, so all tool access runs under that org's RLS (identity model A1) — no new policy surface, no service-role. Thirteen versioned `paw_v1_*` tools return PAW-shaped objects (products, assets, work, estimates, schedule, workflow, asset timeline) plus three lightweight, actor-attributed writes (risk flags, human-review requests, estimate-adjustment proposals) that create paper-trail records — never mutating production state — surfaced in the app's Asset viewer **Agent** tab and the home **Agent activity** widget. Read-heavy by design; the surface deliberately excludes any raw-

query/SQL tool. Per-credential token-bucket rate limiting. Scenario-planner tools and a `submit_work_record` write are planned but not yet built. Design + phase history: [docs/plans/mcp-server.md](https://docs.plans/mcp-server.md).

Synthetic Data Generator

An admin-only tool for populating an Airtable base with realistic-looking test data. Studios save a "target" (an Airtable base with a $P \rightarrow A \rightarrow W$ table mapping and encrypted PAT), then generate Products, Assets, and Work records in configurable counts with optional link fields and extra randomly-populated fields. Records are written in batches of 10 with rate-limit handling. Used to build test environments without real production data.

Platform Admin

System-wide controls for ArtHound operators. Access is restricted to email addresses in the `PLATFORM_ADMIN_EMAILS` environment variable. It currently controls the registration gate: `registration_invite_required` (boolean) and `registration_invite_code` (the platform-wide code new users must enter). A simple admin panel UI at `/admin` exposes these settings.

Web Analytics

Real visitor traffic to both public surfaces, the landing page (`arthound.io`) and the app (`app.arthound.io`), is measured with a self-hosted, cookieless Umami instance at `analytics.arthound.io` . Tracking is client-side only, so the FastAPI backend is not in the data path and the separately deployed static landing page is measured the same way as the app, through one dashboard. Umami runs on Railway with its own dedicated Postgres and Valkey, kept separate from the Supabase production database so analytics data never shares a store with client production data. Each site injects the tracker from a small inline guard that only fires on the real production host, so local and preview traffic never pollute the stats, and no raw IP is ever stored: Umami derives a daily visitor hash and discards the address. Because the tracker is JavaScript and Umami also filters known bot user-agents, the dashboard reflects real human visitors rather than raw requests.

Dev Setup

Backend (from repo root):

```
uvicorn main:app --reload --port 8000
```

Frontend (from `frontend/`):

```
npm run dev      # Vite dev server on :5173, proxies /api to localhost:8000
npm run build    # Outputs to frontend/dist/ (served by FastAPI in prod)
```

Both servers must run simultaneously in development.

Database

Migrations live in `supabase/migrations/` and are applied in filename order. Apply them explicitly and review the diff before running against production; never blind-push to prod, and never apply schema changes through the Supabase dashboard.